UNIDAD #7: SOPORTE Y SEGURIDAD DEL SISTEMA.

El soporte y la seguridad del sistema garantizan la operatividad, disponibilidad y protección de la infraestructura tecnológica, el soporte resuelve fallas y asiste a los usuarios, mientras que la seguridad resguarda los datos y previene accesos no autorizados.

7.1. Ciberseguridad Post-Despliegue

La seguridad no se añade al final como un parche; se hereda del diseño. En producción, el sistema se defiende en dos frentes simultáneos: el código de la aplicación y la infraestructura que lo aloja.

- **Código:** En la mayoría de los casos, los atacantes no hacen "magia" para vulnerar el sistema, buscan errores de lógica dentro de los lenguajes de programación, algunas de estas vulnerabilidades pueden ser:
  a. **Control de Acceso Roto (IDOR - Insecure Direct Object Reference):** Ocurre cuando el backend expone una referencia a un objeto interno de la base de datos directamente en una interfaz (como un ID en la URL o en el cuerpo de un JSON) y confía ciegamente en que el cliente solo pedirá lo que le corresponde.
     - **Vulneración:** Un usuario legítimo inicia sesión con su cuenta (ID: 450). Al ver su perfil, la app web hace una petición a **/api/v1/usuarios/450**. El usuario altera el número en la herramienta de desarrollador del navegador a **/api/v1/usuarios/451**. Si el servidor devuelve los datos del otro usuario solo porque el token de sesión es válido, el sistema está roto.
     - **Mitigación:** La autenticación (saber quién eres) no implica autorización (saber qué puedes hacer), el backend debe validar mediante una consulta intermedia si el ID de usuario extraído de la sesión coincide estrictamente con el ID del propietario del recurso solicitado (peticion.usuario_id ==

recurso.propietario_id), impidiendo que un rol genérico de 'cliente' pueda leer datos de terceros.

b. **Inyección de Datos y el quiebre de contexto:** Ocurre cuando datos no confiables provenientes del usuario son enviados a un intérprete (SQL, NoSQL o la terminal del sistema) como parte de un comando o consulta. El intérprete confunde el *dato* con una *instrucción*.
    - **Mitigación:** El uso de ORMs (como *Prisma* en TypeScript o *Diesel/SQLx* en Rust) que implementan de forma nativa **consultas preparadas (Parameterized Queries)**. Esto separa el mapa de la instrucción SQL de los datos proporcionados por el usuario, haciendo que el motor de la base de datos trate cualquier intento de inyección estrictamente como texto inofensivo.

Muchas organizaciones ofrecen recompensas significativas por el encuentro de este tipo de fallos, a este grupo de personas que buscan errores que puedan representar peligros para el sistema y reportarlos, se les conoce como hacker de sombrero blanco.

- **Infraestructura:** Un código perfecto en un servidor mal configurado es un sistema vulnerable.
    a. **Gestión de Secretos en Entornos de Producción:**
        - **El Riesgo:** Almacenar credenciales de bases de datos, llaves de APIs o secretos de JWT dentro del código fuente o en archivos .env expuestos en repositorios de Git.
        - **La Práctica Profesional:** El código debe configurarse para leer los secretos estrictamente desde las variables de entorno del sistema operativo o del orquestador de contenedores (Docker Secrets / Environment Variables en tiempo de ejecución), garantizando que las llaves reales jamás toquen el repositorio
    b. **Aislamiento de Red mediante Capas:**

- **El concepto:** Un sistema web en producción no debe tener todos sus componentes expuestos a Internet.
- **La configuración:** El contenedor del Frontend y el Proxy Inverso (ej. Nginx) se exponen al mundo exterior (Puertos 80/443). El contenedor del Backend y el de la Base de Datos deben vivir en una red interna virtual privada. La base de datos jamás debe mapear su puerto al host (no debe existir un ports: - 5432:5432 hacia afuera), bloqueando de raíz cualquier intento de ataque por fuerza bruta directo al motor de datos desde Internet, al aislar la base de datos del mundo exterior, los contenedores se comunican entre sí utilizando el DNS interno de Docker a través de una red compartida.

7.2. Soporte, seguimiento y observabilidad: El Hilo de Ariadna.

Cuando un sistema falla en producción, el peor método de soporte es modificar código a ciegas ("adivinar"). El soporte profesional se basa en la Observabilidad: la capacidad de deducir el estado interno de un sistema analizando exclusivamente sus salidas externas.

- **La Gestión Administrativa del Incidente (SLA y Escalado):** Para que el soporte sea sostenible, se estructura en niveles operativos (Mesa de Ayuda / ITIL resumido).
  a. **Nivel 1 (L1 - Soporte Básico):** Recepción del ticket. Filtra errores humanos del usuario (ej. "olvidé mi contraseña").
  b. **Nivel 2 (L2 - Soporte Técnico/Sistemas):** Análisis de infraestructura. Verifica si el servidor está activo, si hay espacio en disco o si falló un servicio.
  c. **Nivel 3 (L3 - Ingeniería/Desarrollo):** El equipo que construyó el software. Interviene cuando el problema requiere una corrección de código (Bug Fix) o un parche de seguridad.
  d. **SLA (Service Level Agreement):** El contrato que obliga al equipo a responder. Si el SLA dice que el tiempo de recuperación (MTTR) para fallos críticos es de 2 horas, el equipo

técnico debe tener las herramientas listas para diagnosticar el problema en minutos.

- **La trazabilidad técnica.**

  Para evitar buscar "la aguja en el pajar" en un archivo de texto plano gigante, se aplican dos estándares de la industria:

  a. **Logs Estructurados (JSON Logging):** En lugar de escribir texto plano que las máquinas no pueden filtrar (logger.info("El usuario entró");), el sistema genera logs en formato JSON en la salida estándar (stdout).

     Ejemplo: {"timestamp": "2026-05-24T18:30:00Z", "level": "error", "module": "auth", "message": "Database connection timeout", "code": "DB_504"}. Esto permite usar herramientas de filtrado rápido en el servidor para aislar solo los eventos de nivel ERROR o CRITICAL en segundos.

  b. **El Identificador de Correlación:**

     - **El Problema:** Cientos de usuarios usan la app al mismo tiempo. Los logs de todos se mezclan en el servidor. Si el usuario Juan da click en "Pagar" y el sistema falla, ¿cómo encuentras los logs específicos de la acción de Juan entre millones de líneas?
     - **La Solución técnica:** Cada vez que una petición entra al servidor, un *middleware* le asigna un identificador único global (un UUID, por ejemplo: req_9b1deb4d). Ese ID viaja a través de todas las capas (Autenticación -> Controlador -> Servicio -> Base de Datos). Si la petición falla, el backend le muestra al usuario el mensaje seguro: *"Ocurrió un error. Reporte el código: req_9b1deb4d"*.
     - El ingeniero de soporte no busca código; busca el **Correlation ID** en el archivo de logs y ve, de forma lineal

y limpia, exactamente qué hilos tiró esa petición específica antes de romperse.

7.3. El Enfoque "Doc-as-Code": RUNBOOK.md

Un sistema que no está documentado para emergencias no puede ser implantado de manera segura, aquí entra el archivo RUNBOOK.md. Este documento es la guía de primeros auxilios que cualquier ingeniero de guardia debe seguir mecánicamente ante un desastre.

````
# Runbook de Operaciones y Emergencias - [hola genesis]

## 1. Diagnóstico de Salud (Triage Inicial)
Pasos para verificar si el sistema está operando correctamente:
1. URL del Health Check: `https://api.proyecto.local/health`
2. Comandos mínimos para verificar recursos en el servidor:
```bash
docker ps  # Verificar contenedores activos
df -h      # Verificar espacio en disco de la Base de Datos)
```

## 2. Protocolo ante Caídas del Servicio (Soporte L2)
Si el endpoint de salud responde 500 o no responde, ejecute los siguientes pasos en orden estricto:
1. Revise los últimos 50 logs críticos del contenedor de la API:
```bash
docker logs --tail 50 genesis_backend | grep -i error
```

2. Si el fallo es por bloqueo de memoria o hilos, ejecute un reinicio limpio y controlado:
```bash
docker compose restart genesis_backend
```

## 3. Plan de Recuperación ante Desastres (Disaster Recovery)
Si la base de datos se corrompe o hay pérdida total de datos, proceda con la restauración del último respaldo (Regla 3-2-1):
1. Ubique el último backup válido en el directorio de almacenamiento seguro `/var/backups/db/.`
2. Detenga el tráfico de la aplicación web de forma temporal.
3. Ejecute el comando de restauración (ejemplo para PostgreSQL):
```bash
cat backup_2026_XX_XX.sql | docker exec -i genesis_db psql -U postgres -d produccion
```
````



Runbook de Operaciones y Emergencias - [hola_genesis]

1. Diagnóstico de Salud (Triage Inicial)

Pasos para verificar si el sistema está operando correctamente:

1. URL del Health Check: `https://api.proyecto.local/health`
2. Comandos mínimos para verificar recursos en el servidor:

```
docker ps  # Verificar contenedores activos
df -h      # Verificar espacio en disco de la Base de Datos)
```

2. Protocolo ante Caídas del Servicio (Soporte L2)

Si el endpoint de salud responde 500 o no responde, ejecute los siguientes pasos en orden estricto:

1. Revise los últimos 50 logs críticos del contenedor de la API:

```
docker logs --tail 50 genesis_backend | grep -i error
```

2. Si el fallo es por bloqueo de memoria o hilos, ejecute un reinicio limpio y controlado:

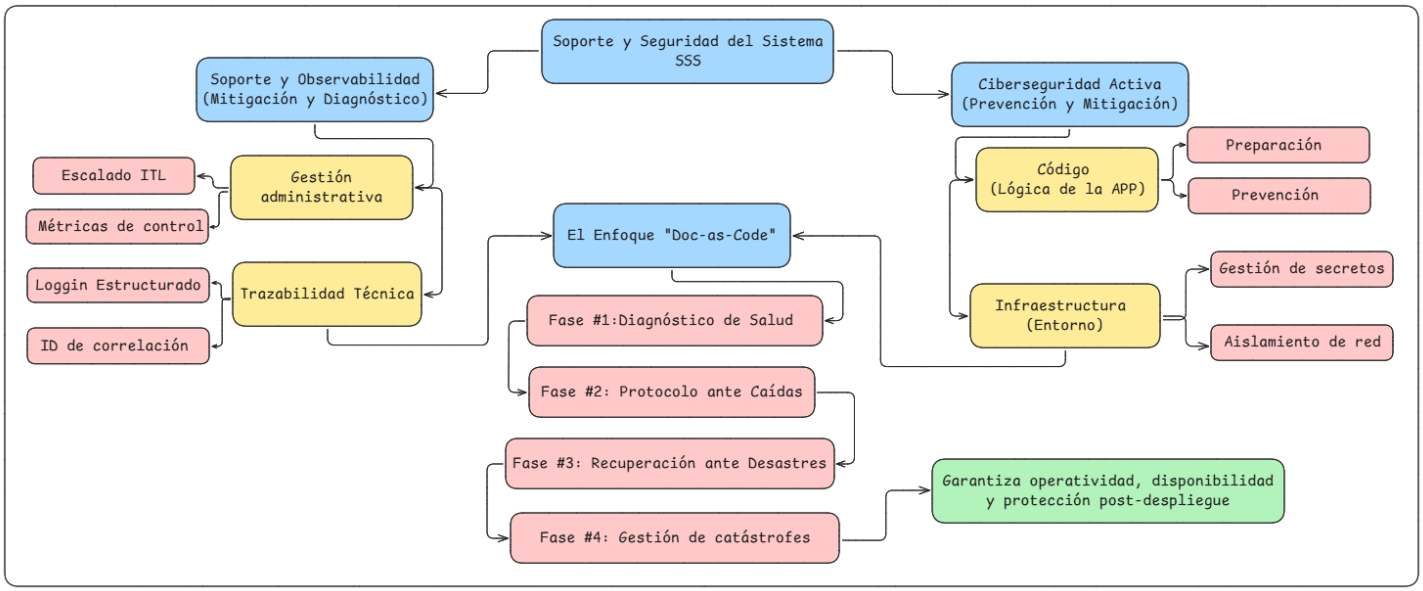
```
docker compose restart genesis_backend
```

3. Plan de Recuperación ante Desastres (Disaster Recovery)

Si la base de datos se corrompe o hay pérdida total de datos, proceda con la restauración del último respaldo (Regla 3-2-1):

1. Ubique el último backup válido en el directorio de almacenamiento seguro `/var/backups/db/.`
2. Detenga el tráfico de la aplicación web de forma temporal.
3. Ejecute el comando de restauración (ejemplo para PostgreSQL):

```
cat backup_2026_XX_XX.sql | docker exec -i genesis_db psql -U postgres -d produccion
```


Soporte y Seguridad del Sistema SSS
Soporte y Observabilidad (Mitigación y Diagnóstico)
Ciberseguridad Activa (Prevención y Mitigación)
Escalado ITL
Gestión administrativa
Métricas de control
Loggin Estructurado
Trazabilidad Técnica
ID de correlación
El Enfoque "Doc-as-Code"
Fase #1:Diagnóstico de Salud
Fase #2: Protocolo ante Caídas
Fase #3: Recuperación ante Desastres
Fase #4: Gestión de catástrofes
Código (Lógica de la APP)
Preparación
Prevención
Infraestructura (Entorno)
Gestión de secretos
Aislamiento de red
Garantiza operatividad, disponibilidad y protección post-despliegue